

Encoder-Level Lipschitz Regularization in STEM-GNN: Preliminary Results

Internal Research Memo
Working draft for discussion

August 27, 2026

Abstract

STEM-GNN’s finetune stage previously regularized only the linear classification head’s Lipschitz constant (via a Frobenius-norm penalty on its weight matrix, `decoder_jac_coeff`), while the GNN encoder beneath it remains unfrozen and unconstrained during finetuning. This memo reports a preliminary comparison of that head-only penalty against a new matching penalty applied to the encoder backbone’s own weight matrices (`encoder_lip_coeff`), added directly to `model/encoder.py`. Results are from a reduced-budget run on Cora and should be read as a first look, not a final claim.

1 Motivation

The finetune objective already included a Jacobian-style penalty on the decoder,

$$\mathcal{L}_{\text{head}} = \lambda_{\text{dec}} \|W_{\text{dec}}\|_F^2,$$

intended to bound the sensitivity of the final linear map to its input. However, during finetuning the encoder itself is not frozen (only the vector-quantizer is, by default) and carries no such constraint. An encoder that is free to amplify its input by an arbitrary factor can pass a perturbation through to the head no matter how tightly the head’s own weight norm is controlled, so a head-only penalty bounds only the last link in the chain. We added an analogous penalty over the encoder’s backbone weight matrices,

$$\mathcal{L}_{\text{enc}} = \lambda_{\text{enc}} \sum_{\ell} \|W_{\ell}\|_F^2,$$

summed over every weight matrix in the backbone’s message-passing layers (SAGE/GAT/GCN/GIN, including per-expert weights when MoE layers are used), excluding biases and GAT’s attention vectors. Both penalties are gated by independent coefficients and default to 0 (opt-in, no behavior change unless requested).

2 Experimental Setup

Honesty note on scope. This is a reduced-budget, single-dataset sanity check, not the project’s standard evaluation protocol. It was run on a single CPU core (no GPU available in this environment), which made the project’s default finetune budget (1000 epochs, early-stop patience 200, 10 splits per dataset, per config) impractical to run interactively. Numbers below should be treated as preliminary evidence about training dynamics, not final benchmark results.

- **Dataset:** Cora, node classification, all 10 splits shipped with the dataset (`finetune.py` always iterates over every provided split; there is no independent random seed here).
- **Backbone:** non-MoE GraphSAGE encoder (2 layers, hidden dim 768), loaded from the `default` pretrain checkpoint at epoch 50 (`ckpts/pretrain_model/default`).

- **Budget:** 60 finetune epochs, early-stop patience 30 – far short of the config-file default (1000 epochs / patience 200). Learning rate and all other hyperparameters use the `config/finetune.yaml` Cora defaults.
- **Coefficients:** a single untuned value, 10^{-3} , for `decoder_jac_coeff` and/or `encoder_lip_coeff` in each configuration below. No sweep over coefficient magnitude was performed.
- **Metric:** node-classification accuracy (%), mean $\pm$ standard deviation across the 10 splits, reported at each split’s best validation epoch (the project’s existing early-stopping selection rule).

3 Results

Configuration	λ_{dec}	λ_{enc}	Train	Val	Test
Baseline (no regularization)	0	0	98.43 $\pm$ 4.25	76.28 $\pm$ 3.34	75.87 $\pm$ 2.10
Head-only (prior behavior)	10^{-3}	0	98.43 $\pm$ 4.25	76.04 $\pm$ 3.35	75.71 $\pm$ 2.30
Encoder-only (new)	0	10^{-3}	99.64 $\pm$ 0.86	75.98 $\pm$ 2.82	75.78 $\pm$ 1.94
Head + Encoder (new, combined)	10^{-3}	10^{-3}	99.93 $\pm$ 0.21	75.96 $\pm$ 2.84	76.03 $\pm$ 0.88

4 Reading the Results

Clean-accuracy means are flat. All four configurations land within roughly one standard deviation of each other on test accuracy (75.7%–76.0%). At this coefficient and epoch budget, neither penalty moves mean clean accuracy on Cora in either direction. This is expected: Lipschitz regularization is a robustness/sensitivity control, not principally an accuracy lever, and clean i.i.d. accuracy is not the metric that should reveal its effect.

The head-only penalty changes nothing measurable. The head-only row is nearly identical to the baseline on every column, including the across-split standard deviation (train std 4.25 $\rightarrow$ 4.25, test std 2.10 $\rightarrow$ 2.30). This matches the original concern that motivated this change: constraining only the decoder’s weight norm has little effect when the unconstrained encoder is free to reshape the input beforehand.

The encoder-side penalty visibly changes training dynamics. Both configurations with $\lambda_{\text{enc}} > 0$ show a sharp drop in across-split variance on train accuracy (std 4.25 $\rightarrow$ 0.86 $\rightarrow$ 0.21) and a smaller but consistent drop in test-accuracy variance (std 2.10–2.30 $\rightarrow$ 1.94 $\rightarrow$ 0.88) as the encoder penalty is added and then combined with the head penalty. In other words, encoder-level regularization made the 10 finetuning runs converge more consistently with each other, even though it did not change their average outcome at this coefficient. That is a real, measured difference in training behavior attributable specifically to constraining the encoder, since the head-only run does not reproduce it.

This does not yet test the actual robustness claim. The motivation for encoder-level Lipschitz control is bounded sensitivity to input perturbations (missing features, dropped or shifted edges, degree/homophily shift) – not clean-split variance. STEM-GNN already ships scripts for exactly this (`scripts/missing_feature.py`, `scripts/random_edge_drop.py`, `scripts/degree_shift_ood.py`, `scripts/homophily_shift_ood.py`), and none of them were run for this memo due to the single-CPU compute budget available. The variance reduction observed here is suggestive but not a substitute for that evaluation.

5 Baseline Hyperparameter Optimization (Optuna)

The comparison in Section 3 uses the un-tuned `config/finetune.yaml` defaults for Cora (learning rate 5×10^{-4} , dropout 0.15, batch normalization) for every configuration. Before trusting any accuracy difference between the regularization settings, it is worth first checking whether the *baseline* model (no head or encoder regularization, i.e. $\lambda_{\text{dec}} = \lambda_{\text{enc}} = 0$) is even reasonably tuned. We used Optuna to search the baseline’s finetune-stage hyperparameters.

What was tuned. The pretrained encoder and vector-quantizer architectures are fixed by the loaded checkpoint (weight shapes must match the saved `state_dict`), so only finetune-stage hyperparameters that do not change layer shapes are searchable without re-pretraining:

- **Learning rate** (`finetune_lr`), log-uniform on $[10^{-4}, 10^{-2}]$;
- **Dropout** (`dropout`), uniform on $[0, 0.5]$;
- **Normalization mode** (`normalize`), categorical over `{none, batch}` (`layer` was excluded – it is accepted by the CLI but the encoder’s forward pass applies `BatchNorm1d` regardless of which string is passed, so it is not actually a distinct option in the current code).

Both regularization coefficients were held fixed at $\lambda_{\text{dec}} = \lambda_{\text{enc}} = 0$ throughout this search: the goal here is a better-tuned *baseline*, not a joint search over baseline and regularization strength.

Implementation. `results/hpo/optuna_baseline.py` drives the search. It builds the same parameter dictionary that `finetune.py`’s command-line entry point would build (Cora defaults from `config/finetune.yaml`), then imports and calls `finetune.run(params)` directly (in-process, no subprocess spawn) so that each trial reuses the already-imported PyTorch/PyG modules. Optuna is configured with its default TPE sampler and, importantly, persists every trial to an on-disk SQLite database (`results/hpo/optuna_baseline.db`). This makes the search resumable: if the process is interrupted, re-running the same script loads the existing study and only runs the remaining trials rather than starting over. *(This safeguard was added after two earlier long-running background experiments in this session were silently killed by the sandbox environment recycling mid-run, losing all unsaved progress.)*

Search budget. Running the full 10-split, 1000-epoch protocol per trial is not tractable on the single CPU core available here, so the search phase uses a reduced budget: a single fixed Cora split (`finetune_seed=0`), 30 finetune epochs, early-stop patience 15, for 12 trials, optimizing validation accuracy on that split. The best-found hyperparameters are then re-evaluated once under the same protocol as Section 3 (all 10 splits, 60 epochs, early-stop patience 30) to get a number directly comparable to the baseline row of that table.

Results. *Pending — the search was still running at the time of writing. This section will be updated with the best hyperparameters found and the resulting all-splits test/val/train accuracy (compared against the untuned baseline row, 75.87 ± 2.10 test / 76.28 ± 3.34 val / 98.43 ± 4.25 train) as soon as it completes.*

6 Next Steps

1. Run the existing OOD/perturbation scripts (missing-feature, random edge-drop, degree/homophily shift) comparing baseline vs. encoder-only vs. combined regularization, which is the setting that should actually distinguish the two penalties.
2. Sweep `encoder_lip_coeff` (e.g. 10^{-4} – 10^{-2}) instead of a single untuned value, since the variance-reduction effect observed here may strengthen or trade off against accuracy at a different magnitude. (A first attempt at this sweep was lost to the same sandbox interruption noted in Section 5 and has not yet been re-run.)
3. Once the Section 5 baseline is tuned, repeat the same Optuna search with λ_{enc} included as a search dimension, to compare a tuned regularized model against a tuned baseline rather than an untuned one.
4. Repeat on a second dataset (e.g. PubMed or WikiCS) and, compute permitting, restore the project’s standard budget (1000 epochs, patience 200) to confirm the trend survives full convergence rather than being an artifact of early stopping at 60 epochs.